

Twenty-Fifth Implementation Iteration: Add ExamRequest

with Explicit Representation of Exam-Based Requests

Software Design / Iteration Report

Name: Najib Attar

Date: May 2026

1. Iteration Goal

The goal of this twenty-fifth implementation iteration is to introduce `ExamRequest` as a first-class request subtype in the resource allocation system.

Before this iteration, exams could only be represented indirectly by using a normal `OneTimeRequest` together with metadata such as a title and a purpose equal to `Exam`. Although that approach was usable for simple testing, it did not express the domain meaning of an exam clearly, and it did not provide a dedicated place for future exam-specific fields and behavior.

In this iteration, exams are modeled explicitly through a separate `ExamRequest` class. An exam is still one-time in terms of schedule because it occurs at a single time slot, but it is not treated merely as a normal one-time request. This iteration therefore creates a cleaner basis for future work such as multi-room exam allocation and controlled sharing of classroom pools, while preserving the current allocation behavior.

2. Changes from Iteration 24

Compared with the twenty-fourth iteration, the following changes were introduced:

- Added a new request subtype: `ExamRequest`
- Added exam-specific fields:
 - `courseCode`
 - `courseName`
 - `examType`
 - `canSplitAcrossRooms`
 - `examTimeSlot`

- Updated `RequestFactory` so that `requestType = "Exam"` creates an `ExamRequest`
- Updated `DataController` to parse optional exam-specific CSV fields
- Added safe defaults for missing optional exam fields:
 - `courseCode = UNKNOWN`
 - `courseName = Unknown Course`
 - `examType = General`
 - `canSplitAcrossRooms = false`
- Updated `AvailabilityRule`, `RuleEngine`, `RuleEngineFacade`, allocation strategies, and `AllocationService` so that `ExamRequest` can be processed safely
- Updated `ConsolePrinter` to display exam-specific details
- Updated `RequestResultWriter` to export exam-specific fields
- Extended `requests.csv` with valid and invalid exam examples

This iteration does not introduce multi-room allocation, a request-type authorization rule, a sharing-aware strategy, or exam-specific optimization logic.

3. Scenario

The prototype still begins by loading users, spaces, requests, and configuration data through the existing controller-based workflow. During request loading, `DataController` continues to parse CSV fields and resolve `User*` and `Space*` references. In addition to the earlier metadata fields, it now also reads optional exam-specific columns such as course code, course name, exam type, and whether the exam may later be split across multiple rooms.

After the row is parsed, `DataController` passes the information to `RequestFactory`. If the request type is `Exam`, the factory creates an `ExamRequest`. If exam-specific optional fields are missing, the request still loads safely by using the default values. If the exam time data is malformed, the row is preserved as an `InvalidRequest` rather than causing a crash or being silently ignored.

Once created, `ExamRequest` follows the same general evaluation flow as a one-time request. The rule engine evaluates the common request rules, the availability rule checks the exam time slot, and the selected allocation strategy either approves the request and creates one allocation or rejects it with a reason. The request is then printed and exported together with its lifecycle history and exam-related details.

4. Modules and Their Tasks

A. Request Base and Subtype Module

- Continue defining the common request structure through the abstract `Request` base class
- Extend the hierarchy with `ExamRequest` as a distinct subtype
- Keep one-time, recurring, exam, and invalid requests represented separately

B. ExamRequest Module

- Store exam-specific information such as course code, course name, exam type, splitting flag, and exam time slot
- Represent an exam as a one-time scheduled event with special domain meaning
- Provide a clean extension point for future exam allocation behavior

C. RequestFactory Module

- Continue creating concrete request objects from parsed input
- Add support for creating `ExamRequest`
- Preserve creation of `OneTimeRequest`, `RecurringRequest`, and `InvalidRequest`

D. DataController Module

- Continue coordinating CSV parsing and reference lookup
- Parse optional exam-related columns when present
- Pass parsed data to `RequestFactory` without directly constructing concrete request objects

E. Rule and Strategy Processing Module

- Continue applying the common request rules through the existing rule engine
- Treat `ExamRequest` like a one-time request for availability and allocation purposes
- Preserve both Greedy and Priority strategies

F. Reporting and Export Module

- Print exam-specific request information in console output
- Export exam-specific fields in `request_results.csv`
- Preserve request status, rejection reason, and lifecycle history output

5. ExamRequest Design

The key design decision in this iteration is that an exam is treated as a one-time event in schedule but as a specialized entity in domain meaning.

This means `ExamRequest` is not implemented by reusing `OneTimeRequest` directly. Instead, it is introduced as a separate subclass of `Request`. The reason is that exams carry information that does not belong to ordinary one-time requests, such as course code, course name, exam type, and a future-oriented split-across-rooms flag.

At the same time, the current prototype does not yet give exams different allocation behavior. For now, `ExamRequest` is evaluated through the same general rules and processed like a one-time request using a single exam time slot. This preserves current system simplicity while making the model more extensible for future exam-specific iterations.

6. Iteration Comparison

Iter.	Focus	Added / Modified	Design bility	Responsi-	Prototype Effect
21	Request priori- tization	PriorityAllocationStrategy	Implemented a separate strategy for deciding processing order.		Requests were no longer processed only by insertion order; priority affected execution.
22	Polymorphic users	User hierarchy and UserFactory	Moved user-specific data such as role, priority, and authorization into user classes.		System behavior became user-driven and easier to extend for different user types.
23	Request cre- ation	RequestFactory	Separated request construction from DataController.		The controller became cleaner and less responsible for object creation.
24	Request meta- data	title and purpose in Request	Extended requests with descriptive information for future use cases.		Requests became clearer, more informative, and better prepared for exam-oriented extensions.
25	Explicit exam requests	ExamRequest and exam-related request fields	Separated exam-specific meaning from generic one-time requests while keeping request creation in RequestFactory.		Exams became first-class requests and the prototype became better prepared for future multi-room exam allocation work.

7. Updated Class Diagram

Figure 1 shows the updated class diagram. The main structural change in this iteration is the addition of `ExamRequest` as a new subclass of `Request`. It also shows that `RequestFactory` now creates `ExamRequest` in addition to the existing request subtypes.

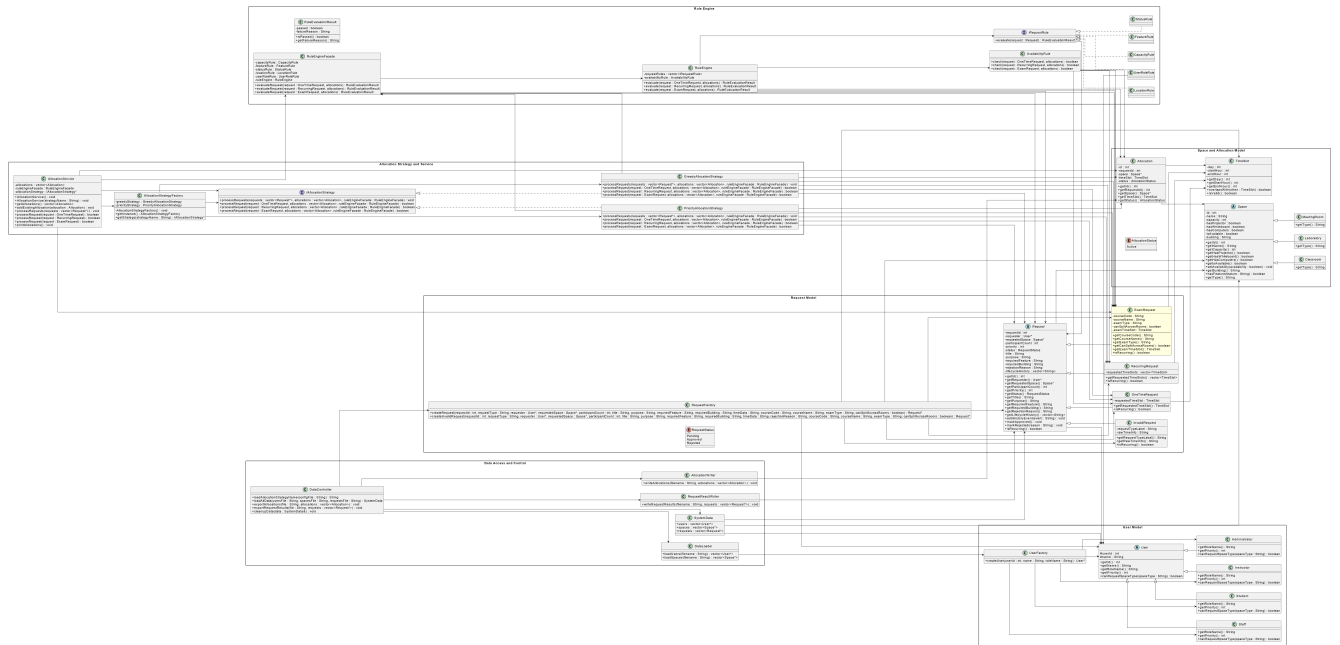


Figure 1: Updated Class Diagram with ExamRequest

8. Updated Workflow Diagram

Figure 2 shows the updated workflow. During request loading, `DataController` now parses optional exam-related fields, and `RequestFactory` creates an `ExamRequest` when the request type is `Exam`. The request is then evaluated and processed like a one-time request for the current iteration.

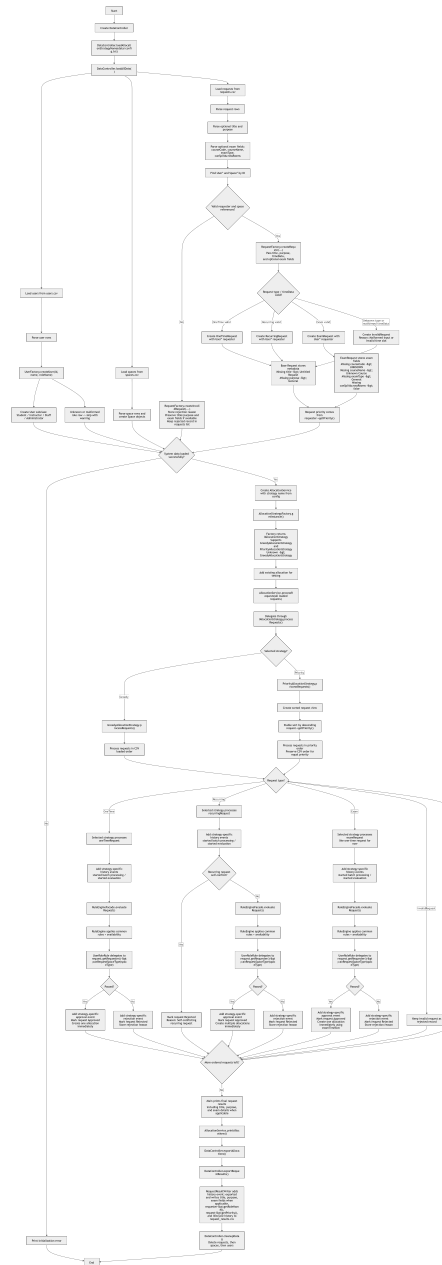


Figure 2: Workflow with ExamRequest Creation and Processing

9. Prototype Implementation

The C++ prototype was extended by adding a new `ExamRequest` class that inherits from the abstract `Request` base class. This class stores both the general request metadata introduced earlier and the new exam-specific fields required in this iteration. It also stores a single exam time slot because an exam remains a one-time scheduled event in the current prototype.

To preserve the current architecture, `RequestFactory` was updated rather than moving exam creation into the controller. When the request type is `Exam`, the factory parses the one-time style time data and creates an `ExamRequest`. If the time data is malformed, the factory safely returns an `InvalidRequest`. Optional exam metadata is accepted when present, and safe defaults are used when it is missing.

`DataController` remains responsible for parsing CSV rows, finding users and spaces, and forwarding the extracted values to the factory. This keeps the separation of responsibilities consistent with the design of the previous iterations. At the same time, `AvailabilityRule`, `RuleEngine`, `RuleEngineFacade`, `AllocationService`, and the existing strategies were updated just enough so that an exam can be evaluated, approved, rejected, and allocated using the same one-slot processing model as a one-time request.

Finally, console output and request result export were extended to include the new exam-specific fields. This means that valid and invalid exam rows can now be inspected clearly during execution and in `request_results.csv`, while existing one-time and recurring requests continue to work without redesign.

10. Summary

This twenty-fifth implementation iteration introduces `ExamRequest` as a first-class request subtype in the resource allocation system. The main purpose of this change is to represent exams explicitly rather than treating them only as ordinary one-time requests with exam-related metadata.

The iteration preserves the current processing behavior by handling exams like one-time requests for now, while adding the structural elements needed for future exam allocation work. As a result, the prototype remains stable and extensible, and it is now better prepared for later iterations involving multi-room exam allocation and related domain-specific behavior.